COS30018 – Intelligent Systems

Task C.4 – Machine Learning 1

This is what I have implemented so far for task C.4:

```
#-----------------------------------------------------------------------------
# Create function to build the model with configurable parameters
#-----------------------------------------------------------------------------
def build_model(
    input_shape,
    layer_type="LSTM",
    layer_sizes=[50, 50, 50],
    dropout_rate=0.2,
    optimizer="adam",
    loss="mean_squared_error",
):
```

First, I created a new function to build the model with these parameters:

- input_shape: I set it later on with this tuple “(PREDICTION_DAYS, n_features)” with the meaning of 60 days. 5 features.
- layer_type: the type of model that the user wants to use, with the default type is set to be “LSTM”.
- layer_sizes: the list of how many layers are there in a model type (the length of the list is the number of layers), and how many units are there in a layer. For example, [50, 50, 50] means that the default LSTM model has 3 layers with 50 units in each layer.
- dropout_rate: the dropout ratio after each layer, I set the “dropout_rate” the default LSTM layer to be 20% or 0.2.
- optimizer: the name of the optimizer you want to have in your model, I set “adam” to be the optimizer for the default LSTM model.
- loss: the name of the loss function you want to have in your model, I set “mean_squared_error” (MSE) to be the loss function for the default LSTM model.

```
#-----------------------------------------------------------------------------
# Map layer type string to the actual Keras layer class
#-----------------------------------------------------------------------------
    layer_map = {
        "LSTM"      : LSTM,
        "GRU"       : GRU,
        "RNN"       : SimpleRNN,
    }

    if layer_type not in layer_map:
        raise ValueError(f"layer_type need to be one of: {list(layer_map.keys())}")
    RecurrentLayer = layer_map[layer_type]
```

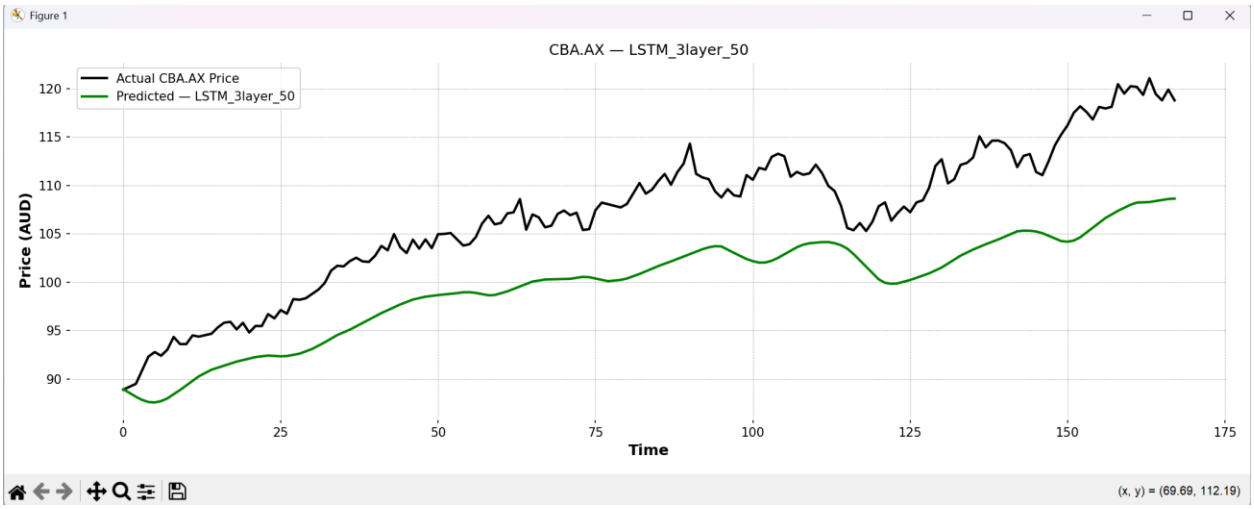
Instead of using “if/else” or “cases”, I implemented this “layer_map” so that I can easily change between the model types, the “RecurrentLayer” can be used as a normal class.

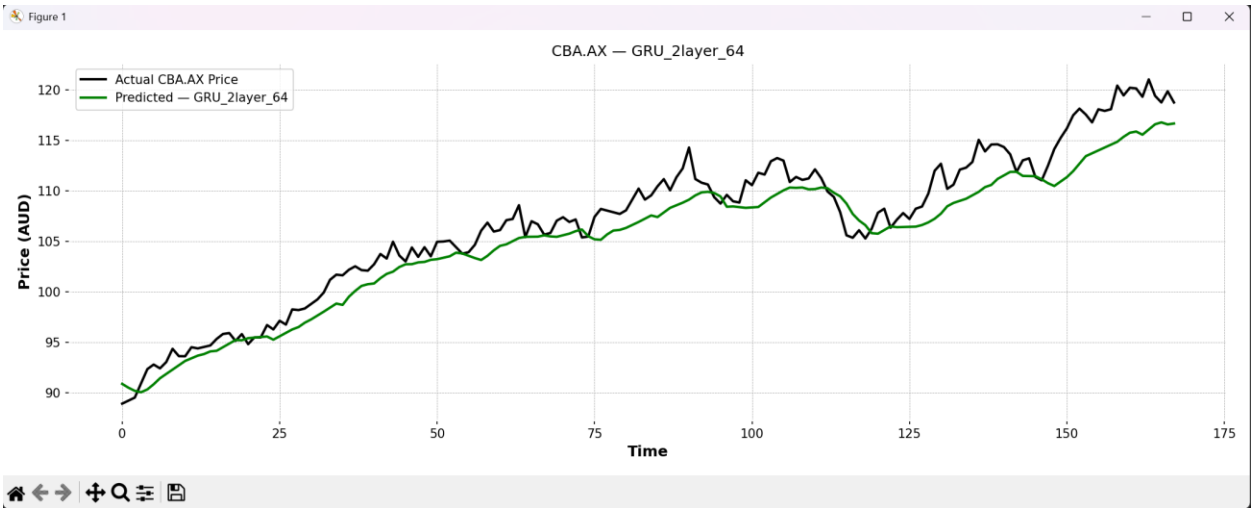
```python
#-----------------------------------------------------------------------------
# Build the Sequential model dynamically
#-----------------------------------------------------------------------------
    model = Sequential()
    for i, size in enumerate(layer_sizes):
        is_first = (i == 0)
        is_last_recurrent = (i == len(layer_sizes) - 1)
        if is_first:
            model.add(RecurrentLayer(
                units            = size,
                return_sequences = not is_last_recurrent,
                input_shape      = input_shape,
            ))
        else:
            model.add(RecurrentLayer(
                units = size,
                return_sequences = not is_last_recurrent,
            ))
        model.add(Dropout(dropout_rate))
```

The next thing I implemented is to make a loop for model building. The loop enumerate using layer_sizes, with each index (i) is the number of units inside each layers. The “is_first” is assigned with index number 0 and it is used to tell if the current layer is the first layer or not, and if it is the first layer, then it will take in the “size” as units of the layer, make the “return_sequences” to return an output string, and also define the input_shape at the first layer. Any layers after the first layer will also return an output string and assign the number of units, but they don’t need to define the “input_shape” anymore, Keras will do it itself. After each layer, it will automatically drop 20% neuron each time it trains to reduce the chance of overfitting.

```python
#-----------------------------------------------------------------------------
# Experiment with different model configurations
#-----------------------------------------------------------------------------
configs = [
    {
        "name"        : "LSTM_3layer_50",
        "layer_type"  : "LSTM",
        "layer_sizes" : [50, 50, 50],
        "dropout_rate": 0.2,
        "optimizer"   : "adam",
        "epochs"      : 25,
        "batch_size"  : 32,
    },
    {
        "name"        : "GRU_2layer_64",
        "layer_type"  : "GRU",
        "layer_sizes" : [64, 64],
        "dropout_rate": 0.3,
        "optimizer"   : "adam",
        "epochs"      : 25,
        "batch_size"  : 32,
    },
    {
        "name"        : "RNN_2layer_32",
        "layer_type"  : "RNN",
        "layer_sizes" : [32, 32],
        "dropout_rate": 0.2,
        "optimizer"   : "rmsprop",
        "epochs"      : 25,
        "batch_size"  : 64,
    },
]
```

I also created a configs which include the setting of 3 different layers including “LSTM”, “GRU”, and “RNN”. I also changed the original code so that now it will run a loop for each config to train, visualize, and predict the next day of those 3 different layers. Here are the results of what happen after I run the program:





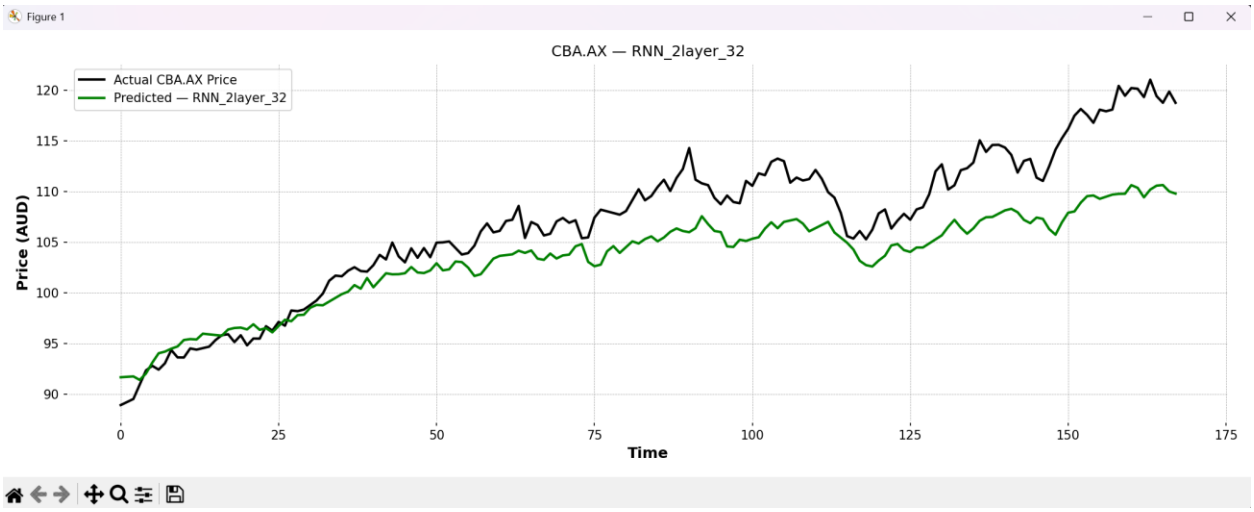


Figure 1
CBA.AX — RNN_2layer_32
Actual CBA.AX Price
Predicted — RNN_2layer_32
Price (AUD)
120
115
110
105
100
95
90
0
25
50
75
100
125
150
175
Time